

Process / Thread Core

- **Process**: running program + address space + resources + kernel bookkeeping.
- **Thread**: execution stream inside a process. Own PC, registers, stack, state; shares process address space/files.
- **PCB/TCB**: kernel records for process/thread: state, registers, scheduling info, address-space info, open files.
- **Kernel**: privileged OS core. **System call**: controlled user -> kernel entry.
- **Context switch**: save current thread state, restore next thread state.
- Thread states: running, ready, blocked. Ready means runnable but not on CPU.

Process/FD API Must-Know

call	returns	exam use
pid_t fork(void)	0 child, child PID parent, -1 error	create duplicate process
int execvp(char *file, char *argv[])	only returns -1 on failure	replace current process image
pid_t waitpid(pid_t, struct rusage *, int)	child PID or -1	wait/reap exact child
int pipe(int p[2])	0 or -1	p[0] read, p[1] write
int dup2(oldfd, newfd)	newfd or -1	redirect fd newfd to same target as oldfd
int open(path, flags, mode)	new lowest fd or -1	file input/output
ssize_t read/write(fd, buf, n)	bytes, 0 EOF for read, -1 error	raw byte I/O

fork

fork() duplicates the calling process. Both continue after the call.

Where	return value	meaning
child	0	I am the newly created child.
parent	child PID	I created this child.
error	-1	No child created.

- Memory is separate after fork; variables are copied, not shared.
- Open file descriptors are inherited.
- Output order is not guaranteed unless synchronized or waited.
- Loop with N unconditional forks: total processes 2^N, new processes 2^N - 1.

```
pid_t pid = fork();
if (pid == 0) { /* child */ }
else { /* parent; pid is child's PID */ }
```

Multiple Fork Trace

```
pid_t a = fork();
pid_t b = fork();
```

Each existing process executes the next fork. Child of a later fork copies old variable values from its parent.

process	a	b
original parent	PID of first child	PID of its second child
first child	0	PID of its child
second child from parent	PID of first child	0
second child from first child	0	0

execvp

execvp loads a new executable file into the current process. Same PID, new program image.

```
char *args[] = {"wc", "-l", NULL};
execvp(args[0], args);
perror("execvp"); exit(1);
```

- p means search PATH, like shell. "wc" might resolve to /usr/bin/wc.
- Replaces old code, globals, heap, stack. Does not create a new process.
- If successful, never returns to old code.
- argv array must end with NULL.
- File descriptors usually survive across exec unless close-on-exec.

waitpid / Zombies

```
int status;
waitpid(pid, &status, 0);
if (WIFEXITED(status))
    WEXITSTATUS(status);
```

- waitpid(pid, NULL, 0): wait for exact child, ignore status, blocking wait.
- Collects child's exit status and prevents zombie persistence.
- status is encoded; use WIFEXITED, WEXITSTATUS.

Shell Pattern

```
pid_t pid = fork();
if (pid == 0) {
    dup2(fd, STDIN_FILENO);
    close(fd);
    execvp(args[0], args);
    perror("execvp"); exit(1);
}
waitpid(pid, NULL, 0);
```

Parent keeps shell/program alive. Child redirects FDs, then becomes command.

File Descriptors

fd	name	constant
0	stdin	STDIN_FILENO
1	stdout	STDOUT_FILENO
2	stderr	STDERR_FILENO

- open returns lowest available fd.
- read(fd, buf, n). write(fd, buf, n). close(fd).
- dup2(oldfd, newfd): make newfd refer to same open file description as oldfd. Direction matters.

Pipes

```
int fds[2]; pipe(fds);
// fds[0] read end, fds[1] write end
```

- Data written to fds[1] is read from fds[0].
- After fork, both processes inherit both ends.
- EOF only when all write ends are closed. Exam trap: leaked write end -> reader blocks forever.
- Pipe command pattern: child1 stdout -> pipe write; child2 stdin -> pipe read; close unused ends in all processes.

C/C++ Syntax Traps

- p->field means (*p).field.
- &x address of x. *p object pointed to by p.
- Array of strings: char *argv[] = {"ls", "-l", NULL};. array of char pointers; each string ends in '\0'; argv list ends in NULL.
- std::deque<T> is a double-ended queue, not the verb dequeue. push_back add at back, front first, pop_front remove first.

Command-Line Args

```
int main(int argc, char *argv[]) {
    // argv[0] program name
    // argv[1] first real arg
}
```

- argv is an array of C-string pointers. Each string ends with '\0'.
- argc counts entries including argv[0].
- For execvp, argument vector itself ends with NULL, not just '\0'.

FD Table vs Open File

- Process fd table maps small ints to open-file descriptions.
- Open-file description stores file object/inode, mode, current offset, flags.
- fork copies fd table entries; parent/child entries can refer to same open-file description, so file offset can be shared.
- dup2 makes two fd numbers refer to same open-file description.
- close(fd) removes this process's fd entry; file object closes only when last reference gone.

Read/Write Return Values

- read returns bytes read, 0 for EOF, -1 for error.
- write returns bytes written or -1; may be partial, so robust code loops.
- perror("x") prints message using current errno.
- exit(n) terminates process with status n; returning from main is similar.

Process Identity

- getpid(): current process ID. getppid(): parent process ID.
- PID stays same across successful exec.
- fork creates new PID; exec changes program image only.
- Zombie: child exited but parent has not waited. Orphan: parent exits first; OS reparents child.

Pipe Pipeline Skeleton

```
pipe(p);
if (fork()==0) {
    dup2(p[1], STDOUT_FILENO);
    close(p[0]); close(p[1]);
    execvp(cmd1[0], cmd1);
}
if (fork()==0) {
    dup2(p[0], STDIN_FILENO);
    close(p[0]); close(p[1]);
    execvp(cmd2[0], cmd2);
}
close(p[0]); close(p[1]);
waitpid(...); waitpid(...);
```

Parent must close both pipe ends too, or reader may never see EOF.

Process / FD Rapid Fire

- fork copies user-space memory; child gets copied globals/heap/stack values.
- fork does not copy "one thread per parent thread" in the ordinary kernel-thread sense; user-level thread bookkeeping may be copied as memory.
- Parent/child after fork are scheduled independently; either may run first.
- If a variable is changed before second fork, child of second fork inherits that changed value.
- printf before fork can be duplicated if output buffer not flushed; newline/terminal often flushes but do not rely blindly.
- execvp searches PATH; execv uses explicit path.
- After successful exec, old signal handlers/memory/thread state mostly gone; open fds remain unless close-on-exec.
- Close unused pipe ends immediately after dup2.
- dup2(fd, 1) redirects stdout; dup2(1, fd) is the wrong direction for redirecting stdout.
- stderr is fd 2, often unbuffered; useful for errors even when stdout redirected.
- Exit status convention: 0 success, nonzero failure.
- Parent waiting for children in either order is okay unless output/order requirement says otherwise.
- If child should not continue parent loop, it must return/exit after its child work.
- Shell command cmd > file: open file, dup2(file, 1). exec command.
- Shell command a | b: connect a stdout to pipe write, b stdin to pipe read.

Threads / Process Weird Cases

- Multiple threads in one process share globals/heap/fd table; each thread has own stack/registers/PC.
- std::thread t(f); t.join();: parent waits for thread to finish, analogous intuition to wait but within same process.
- Successful exec in multithreaded process replaces entire process image; old threads are gone; new program starts normally.
- User-level threads are represented by data structures in process memory; fork can copy those records as ordinary memory.
- Kernel threads are known/scheduled by kernel; user-level threads may be invisible to kernel scheduler.
- After fork in a multithreaded program, be careful with copied locks/metadata that may describe vanished execution contexts.

Struct / Pointer Syntax

```
typedef struct Point { int x; int y; } Point;
Point p; p.x = 3;
Point *q = &p; q->x = 4; // (*q).x
```

- Point p; declares a struct variable with fields x, y.
- typedef ... Point; creates the convenient type name Point.
- . for object, -> for pointer to object.

Worked: Fork Values

```
pid_t a = fork();
pid_t b = fork();
printf("a=%d b=%d\n", a, b);
```

process	a	b	why
P0	PID(P1)	PID(P2)	parent of both forks
P1	0	PID(P3)	child of first, parent of second
P2	PID(P1)	0	child of second fork from P0; copied old a
P3	0	0	child of second fork from P1

Rule: 0 only in the new child of that exact fork call; old variables are copied from parent.

Worked: Redirect + Exec

```
int fd = open("out", O_WRONLY|O_CREAT|O_TRUNC, 0644);
pid_t pid = fork();
if (pid == 0) {
    dup2(fd, STDOUT_FILENO);
    close(fd);
    char *a[] = {"echo", "hi", NULL};
    execvp(a[0], a);
    exit(1);
}
close(fd); waitpid(pid, NULL, 0);
```

Child becomes echo; same child PID; fd 1 points to out, so hi goes to file. Parent does not exec and waits.

Worked: Correct Pipeline

Implement left | right. Key: fork two children, use dup2, close all inherited pipe ends.

```
int p[2]; pipe(p);
pid_t left_pid = fork();
if (left_pid == 0) {
    dup2(p[1], STDOUT_FILENO); // stdout -> pipe
    close(p[0]); close(p[1]);
    execvp(left[0], left); perror("exec left"); exit(1);
}
pid_t right_pid = fork();
if (right_pid == 0) {
    dup2(p[0], STDIN_FILENO); // stdin <- pipe
    close(p[0]); close(p[1]);
    execvp(right[0], right); perror("exec right"); exit(1);
}
close(p[0]); close(p[1]);
waitpid(left_pid, NULL, 0);
waitpid(right_pid, NULL, 0);
```

If parent keeps p[1] open, right may block waiting for EOF forever.

Worked: Child Loop Trap

```
for (int i=0; i<3; i++) {
    pid_t pid = fork();
    if (pid == 0) {
        do_child_work(i);
        exit(0); // critical: child must not keep forking
    }
}
while (waitpid(-1, NULL, 0) > 0) {}
```

Without exit, each child continues the loop and creates extra children. waitpid(-1, ...) waits for any child.

Race / Critical Section

- **Race condition:** correctness depends on timing/interleaving.
- **Critical section:** shared-state access that must be mutually exclusive.
- Atomicity: operation appears indivisible to other threads.
- Thread-safe: correct under concurrent calls.
- Interrupts alone do not solve multicore races: another core can still run.

```
// x++ is read, add, write; not atomic
int tmp = x; tmp = tmp + 1; x = tmp;
```

Mutex / RAIL

```
std::mutex m;
void f() {
    std::unique_lock<std::mutex> lock(m);
    // critical section
} // auto unlock
```

- Creating `unique_lock/lock_guard` locks the mutex.
- Leaving scope unlocks automatically.
- Use `unique_lock` with condition variables because `wait` must unlock/relock.
- Manual `m.lock()`; ... `m.unlock()`; is error-prone, especially early returns.

Condition Variables

```
std::unique_lock<std::mutex> lock(m);
while (!predicate) {
    cv.wait(lock);
}
// predicate true while lock held
```

- `wait(lock)` atomically releases mutex and sleeps; on wake, reacquires before returning.
- Always use `while`, not `if`: spurious wakeups, stale predicates, wrong waiter woken.
- Change shared state while holding lock, then notify.
- `notify_one`: one waiter can proceed. `notify_all`: multiple/different predicates or simplicity.

Thread/Sync API Must-Know

API	meaning/trap
<code>std::thread t(fn,args)</code>	starts concurrent thread running <code>fn(args)</code> .
<code>t.join()</code>	waits for that thread to finish; call once.
<code>std::mutex m</code>	mutual exclusion object; protects shared state.
<code>std::unique_lock<mutex> lock(m)</code>	locks now, unlocks at scope end; usable with CV.
<code>cv.wait(lock)</code>	atomically unlocks + sleeps; relocks before returning.
<code>cv.notify_one/all()</code>	wakes waiters; predicate still decides who can proceed.
<code>deque.push_back(x), front(), pop_front()</code>	queue selected waiter records in arrival order.

Monitor Pattern

Monitor = mutex + shared state + condition variables + methods that hold lock while checking/updating state.

```
class Buffer {
    mutex m; condition_variable nonempty, nonfull;
    queue<Item> q; int cap;
    void put(Item x) {
        unique_lock<mutex> lock(m);
        while (q.size()==cap) nonfull.wait(lock);
        q.push(x); nonempty.notify_one();
    }
    Item get() {
        unique_lock<mutex> lock(m);
        while (q.empty()) nonempty.wait(lock);
        Item x=q.front(); q.pop();
        nonfull.notify_one(); return x;
    }
};
```

Selected-Thread Grouping

If prompt says exactly selected threads proceed, counters alone are unsafe. Store waiter records.

```
struct Waiter { bool assigned=false; condition_variable cv; };
deque<Waiter> H, O;
void try_group() {
    if (H.size())>=2 && O.size())>=1) {
        Waiter *h1=H.front(); H.pop_front();
        Waiter *h2=H.front(); H.pop_front();
        Waiter *o=O.front(); O.pop_front();
        h1->assigned=h2->assigned=o->assigned=true;
        h1->cv.notify_one(); h2->cv.notify_one(); o->cv.notify_one();
    }
    void hydrogen() {
        Waiter self; unique_lock<mutex> lock(m);
        H.push_back(&self); try_group();
        while (!self.assigned) self.cv.wait(lock);
        lock.unlock(); bond();
    }
}
```

- `h1->cv = (*h1).cv`. Wake exact selected waiter.
- `&self` address of this thread's local waiter record.

Deadlock

Coffman conditions: mutual exclusion, hold-and-wait, no preemption, circular wait.

- Prevent circular wait by global lock ordering.
- Acquire all resources at once, or release before waiting.
- Starvation != deadlock. Starvation: one waits too long though system progresses. Deadlock: cycle, no progress.

```
// order rule
lock(A); lock(B); ... unlock(B); unlock(A);
```

Low-Level Lock Implementation

- Single core: disabling interrupts can protect lock/scheduler internals briefly.
- User mutex does not disable interrupts for your entire critical section.
- Multicore: disabling interrupts on one core is insufficient; need atomic instructions.
- Test-and-set/swap/CAS allow read-modify-write atomicity.
- Busy waiting wastes CPU but can be necessary inside low-level spinlocks, especially very short waits.

Common Sync Bugs

- Wait predicate checked with `if` instead of `while`.
- Notify before changing state.
- Holding lock during slow/external work.
- Sleeping after partially consuming resources.
- Forgetting "in progress" state: boarding/crossing/merging but not finished.
- Using `notify_all` when exactly one specific waiter must proceed, without assignment state.

Monitor Design Checklist

1. List methods and which threads call each.
2. Write shared state fields before code.
3. Write wait predicate as Boolean expression over shared state.
4. Acquire lock at start of each monitor method.
5. Update waiting counts/records before sleeping.
6. Use `while` around every `wait`.
7. Change shared state before notifying.
8. Release lock before slow/external simulated work when safe.

CalTrain Pattern

- State needs seats available, waiting passengers, passengers currently boarding.
- Passenger waits until train present and seat available.
- Train waits until no boarders-in-progress or no seats/waiters remain.
- Trap: count passenger after signaled but before `boarded()`; otherwise train may leave early.

Bridge / Merge Pattern

- Bridge: track current direction, cars on bridge, waiting per direction, capacity.
- Wait while wrong direction, bridge full, or fairness rule says other direction should go.
- Merge: track lane queues, current merging car, fairness/no-starvation rule.
- Trap: "notify a lane" may not identify exact car unless lane order/state is explicit.

Inventory Reservation

- Request needs multiple item counts atomically.
- Wait until all requested items are available at once.
- Do not subtract item A then sleep for item B; that partially reserves and can block others.
- `add()` often uses `notify_all` because different orders have different predicates.

Condition Variable Semantics

- CV has no memory. If nobody is waiting, notification can be lost.
- The real condition is the shared-state predicate, not the CV itself.
- Wait returns with lock held, so recheck predicate immediately while state is protected.
- Signaling thread usually keeps running until it releases the lock; notified thread competes to reacquire.

Priority Inversion

High-priority thread waits for lock held by low-priority thread, while medium-priority work prevents low-priority owner from running. Fix idea: priority donation/inheritance.

Synchronization Rapid Fire

- Protect every shared variable that participates in a predicate with the same mutex.
- A predicate like `count > 0` must be checked while holding the lock protecting `count`.
- `notify_one` does not transfer the lock to the waiter; waiter still must reacquire.
- Unlocking before notify is sometimes okay, but exam-safe pattern is update state then notify while reasoning clearly.
- Condition variables solve sleeping, not mutual exclusion. You still need the mutex.
- Busy waiting: repeatedly checking a condition while consuming CPU.
- Blocking wait: thread sleeps; scheduler runs someone else.
- Semaphore intuition: counter + wait/signal; mutex is binary mutual exclusion conceptually, but use course's monitor style when asked.
- Reader/writer style: many readers allowed if no writer; writer needs exclusive access; fairness policy prevents starvation.
- Barrier style: threads wait until count reaches N, then all proceed; need generation number for reusable barrier.
- Exactly-one/lowest-number/selected-thread prompts require identity/queue state, not only counts.
- All-or-nothing resource prompts require checking all resources before consuming any.
- No-starvation prompts require turn/fairness state, not just "while unsafe wait".
- Spurious wakeup: wait can return even without useful notification; hence `while`.
- Lost wakeup avoided because `wait(lock)` releases lock and sleeps atomically.
- Invariant examples: count never negative; bridge cars $\leq$ capacity; one merge at a time; selected waiters exactly match group.

Atomic Ops / Spinlock Sketch

```
while (test_and_set(&lock) == true) {
    // spin
}
// critical section
lock = false;
```

- Atomic read-modify-write makes check-and-acquire indivisible.
- Spinlocks waste CPU if held long; okay for tiny kernel critical sections.
- On single core, spinning while lock holder cannot run is bad unless interrupts/preemption rules guarantee short wait.
- Interrupt disable protects against preemption on same core, not concurrent execution on other cores.

When To Use What

need	tool/state
one-at-a-time shared update	mutex
sleep until condition	condition variable + predicate
exact selected thread	per-waiter record/CV
group of N	queues + assignment counts
fair alternation	turn/direction/served counters
all resources available	check all before decrement any

Worked: Monitor Predicate

Prompt: "Cars may cross bridge if same direction, capacity 3, no opposite-direction car currently crossing."

```
while ((on_bridge > 0 && dir != my_dir) ||
        on_bridge == 3 ||
        should_yield_to_other_dir(my_dir)) {
    cv[my_dir].wait(lock);
}
on_bridge++;
dir = my_dir;
```

- Shared state: `on_bridge`, current `dir`, wait counts/fairness state.
- Predicate says when this car cannot safely proceed.
- On exit: decrement `on_bridge`, maybe switch direction, notify appropriate waiters.

Worked: Inventory All-Or-Nothing

```
// need: map item -> requested count
while (!all_available(need)) {
    cv.wait(lock);
}
for (auto [item,n] : need) {
    stock[item] -= n;
}
```

Correct because it waits before subtracting anything. Wrong solution subtracts available items, then sleeps for missing items, creating partial reservation/deadlock/starvation risk.

Worked: Lost Update Race

```
int x = 0;
void worker() { x = x + 1; } // two threads
```

step	T1	T2
1	read x=0	
2		read x=0
3	write x=1	
4		write x=1

Final can be 1, not 2. Fix: protect read-modify-write with same mutex.

```
void worker() {
    unique_lock<mutex> lock(m);
    x = x + 1;
}
```

Worked: Reusable Barrier

```
int arrived=0, generation=0; mutex m; condition_variable cv;
void barrier(int n) {
    unique_lock<mutex> lock(m);
    int my_gen = generation;
    arrived++;
    if (arrived == n) {
        arrived = 0; generation++;
        cv.notify_all();
    } else {
        while (my_gen == generation) cv.wait(lock);
    }
}
```

`generation` prevents a thread from the next round passing because of an old notification.

Worked: Deadlock Code

```
// bad
T1: lock(A); lock(B);
T2: lock(B); lock(A);
// good: every thread uses same order
lock(A); lock(B); ... unlock(B); unlock(A);
```

Opposite lock order creates circular wait. Exam fix: global order or acquire all resources atomically in one monitor.

Dispatching vs Scheduling

- Scheduler:** policy deciding which ready thread should run.
- Dispatcher:** mechanism that switches CPU to chosen thread.
- Dispatcher is OS code, not a special high-priority thread.
- Preemption does not release locks; only owning thread can safely release.

Dispatcher/scheduler share ready-queue state; scheduler updates policy/data, dispatcher quickly selects/runs.

Events Causing Dispatch

- Running thread blocks on I/O/condition/page fault.
- Thread exits.
- Timer interrupt/time slice expires.
- Higher-priority thread becomes ready.
- Yield/system call may trigger scheduling.

Scheduling Goals

goal	meaning
throughput	jobs completed/time
turnaround	finish - arrival
response	first run/output latency
fairness	reasonable sharing/no starvation
utilization	keep cores busy

Algorithms

policy	good	bad/trap
FIFO/FCFS	simple	convoy effect; poor response
Round robin	fair, responsive	too small quantum -> switch overhead; too large -> FIFO-like
SRPT/SJF	great average turnaround if estimates good	needs job length estimate; long jobs may starve
Priority	policy/control	starvation, inversion

I/O-bound jobs often get good response under SRPT-like heuristics because CPU bursts are short.

Priority Queues

Implementation: one ready queue per priority level. Dispatcher picks highest non-empty priority queue; ties often round-robin.

```
highest priority: Q0 -> run first
Q1
Q2
lowest priority
```

4.4 BSD Scheduler

- Real Unix scheduler example. Combines priority, recent CPU usage, user hints, interactivity heuristics.
- Dispatcher records start/stop times; scheduler estimates recent CPU usage.
- Rule: least recent CPU usage -> higher priority; lots recent CPU -> lower priority.
- Interactive/I/O-bound: run briefly then block -> low recent CPU -> high priority.
- CPU-bound: uses full slices -> accumulates CPU -> lower priority.
- CPU-bound should not starve forever: while waiting, recent CPU stops increasing/decays, priority rises.
- Overloaded/tied case can devolve to round robin among highest-priority runnable threads.

Unix nice

- nice=0 default.
- +19: very nice, emphasize CPU usage, lower priority.
- 20: least nice, de-emphasize CPU usage, higher priority.
- nice is a hint/input, not absolute CPU control.

```
nice -n 19 ./background_script.sh
nice -n -20 ./urgent
```

Multicore Scheduling

- Shared: ready queues/scheduler data + lock.
- Per core: interrupts, dispatcher, currently running thread.
- Run k highest-priority ready threads on k cores.
- New high-priority ready thread may preempt lowest-priority running thread.
- Need synchronization around scheduler queues because multiple cores access them.
- Work-conserving: do not leave core idle when ready work could run.
- Work stealing: idle core takes work from busier core/queue. Helps load balance.
- Core affinity/locality: running same thread on same core may preserve cache/TLB warmth.

Exam Phrases

- Reducing recent CPU usage, by blocking/sleeping/yielding, can improve BSD effective priority.
- Recent CPU usage, not lifetime CPU usage.
- Preemption can expose interleavings but does not itself make code safe.
- Voluntary yield not required for preemptive scheduler to regain CPU; timer interrupt can do it.

Scheduling Calculations

- Completion time: when job finishes.
- Turnaround: completion - arrival.
- Response: first scheduled - arrival.
- Waiting time: time ready but not running.
- Average metrics: compute per job, then average; do not average endpoints.

Preemptive vs Cooperative

- Cooperative: running thread gives up CPU voluntarily by yield/block/exit.
- Preemptive: timer interrupt lets OS regain control and switch threads.
- Preemption can happen while user code holds a mutex; that mutex remains locked.
- Trap: "dispatcher runs" does not mean a lock is released or shared data becomes safe.

Traps / Interrupts

- Trap: synchronous control transfer caused by current instruction, e.g. syscall/fault.
- Interrupt: asynchronous event, e.g. timer/device.
- Both enter kernel via controlled mechanism and save enough state to resume/handle.
- Timer interrupts are what make preemptive scheduling possible on one core.

MLFQ Intuition

- Multiple feedback queues approximate unknown job length.
- New/interactive jobs start high priority.
- Using whole time slice suggests CPU-bound -> move lower.
- Blocking before slice ends suggests interactive/I/O-bound -> stay high.
- Aging/boosting prevents starvation.

Ready Queue Shared State

- Ready queues are kernel shared state. Lock/scheduler internals need atomic updates.
- Disabling interrupts helps on single core during tiny kernel critical sections.
- On multicore, also need spinlocks/atomic ops because other cores keep running.
- Dispatcher must not be interrupted halfway through inconsistent queue/state updates.

CPU vs I/O Bound

type	behavior	scheduler effect
CPU-bound	long CPU bursts, uses slices	lower BSD priority over time
I/O-bound	short CPU, blocks on I/O	often high priority/responsive
interactive	waits for user input	should get quick response

Scheduling Rapid Fire

- Convoy effect: one long CPU-bound job delays many short/I/O-bound jobs under FIFO.
- Round-robin response improves because jobs get CPU periodically before finishing.
- Quantum tradeoff: shorter improves response but increases context-switch overhead.
- SRPT minimizes average turnaround under ideal known remaining times, but estimates are hard.
- Priority scheduling can express importance but risks starvation.
- Aging gradually raises waiting jobs' priority to prevent starvation.
- Priority inversion is not deadlock; system can progress but priority policy is defeated.
- Work-conserving scheduler runs ready work if any exists; exceptions may involve power/locality/policy.
- Per-core ready queues reduce contention but can imbalance load.
- Global ready queue balances easily but needs shared lock/coordination.
- Thread affinity improves cache/TLB locality but can conflict with load balancing.
- Interrupts insufficient to prevent races on multicore; atomic ops/locks needed.
- Running -> blocked: waits for I/O/lock/CV/page. Blocked -> ready: event occurs. Ready -> running: scheduler picks it.
- Trap/fault/syscall can enter kernel on behalf of current thread; timer interrupt can enter kernel asynchronously.

Gantt Trace Method

- Sort arrivals by time.
- At each decision point, list ready jobs only.
- Apply policy exactly: FIFO order, shortest remaining, highest priority, or RR queue head.
- For preemptive policies, check new arrivals/timer ticks for possible switch.
- Record first-run time and completion time for each job.

BSD Short Answer Template

"BSD uses dynamic priority based partly on recent CPU usage. A CPU-bound thread accumulates recent CPU and drops in priority; an I/O-bound or interactive thread blocks often, accumulates less recent CPU, and stays responsive. Waiting lets priority recover, so CPU-bound work should not starve forever. nice shifts the priority behavior but is only a hint."

Scheduler Bug Reasoning

- A scheduler change can expose a race by making a previously rare interleaving common.
- If code is only correct under one schedule, it is not synchronized correctly.
- Correct concurrent code must tolerate arbitrary preemption at instruction boundaries, except inside properly protected critical sections.

Dispatcher Sequence

- Kernel enters because current thread blocks, exits, traps, or timer interrupt fires.
- Save current thread registers/PC/stack pointer in TCB if it will resume later.
- Move current thread to appropriate state: ready, blocked, or finished.
- Pick next ready thread using scheduling policy.
- Restore next thread's saved registers/PC/stack pointer.
- Return from trap/interrupt into next thread's execution.

Per-Core Scheduling Facts

- Each core has its own current running thread or idle loop.
- A multicore system can have multiple threads running truly simultaneously.
- Locks protect scheduler structures because different cores may enqueue/dequeue at same time.
- Disabling interrupts on core A does not stop core B from modifying shared memory.
- Cross-core preemption may require interrupting another core to reschedule it.

Policy Comparison Phrases

- FIFO favors simplicity and low overhead but can be terrible for interactive response.
- RR approximates fair sharing when all ready threads should make progress.
- SRPT is aggressive about short jobs; great average turnaround, possible unfairness.
- Priority scheduling separates mechanism from policy: priority value encodes desired preference.
- BSD dynamic priority is a heuristic for interactivity without explicitly knowing job type.

Scheduling Mistakes

- Including blocked jobs in ready-queue decisions.
- Forgetting RR requeues unfinished job at back after quantum expires.
- Preempting non-preemptive FIFO/SJF when a shorter job arrives.
- Computing response time from completion instead of first run.
- Assuming all jobs arrive at time 0 when arrivals differ.
- Assuming priority value direction without checking problem statement.

Worked: RR Trace

All arrive at 0. Quantum 2. Bursts: A=5, B=3, C=1.

```
0-2 A rem3
2-4 B rem1
4-5 C done
5-7 A rem1
7-8 B done
8-9 A done
```

job	first run	finish	response	turnaround
A	0	9	0	9
B	2	8	2	8
C	4	5	4	5

Worked: SRPT Decision

At t=0 A needs 8. At t=2 B arrives needing 3. A has 6 remaining, B has 3, so preempt A and run B. SRPT always compares remaining time among ready jobs.

Storage Comparison

device	important asymmetry
magnetic disk	seek/rotation dominate random I/O
SSD/flash	erase before rewrite, wear, FTL remapping
RAM	byte-addressable, volatile, much faster

Disk Request Reasoning

- For SSTF/SCAN questions, track current head position and direction.
- SSTF can repeatedly serve nearby requests and starve distant ones.
- SCAN moves like elevator; C-SCAN wraps to one side to provide more uniform wait.
- Shortest seek is not always best if fairness or starvation matters.

Disk Locality / BSD FS

- Keep related inodes/data near each other to reduce seek time.
- Spreading structures across disk can avoid all metadata being far from data.
- Large blocks improve throughput; fragments reduce internal waste for small file tails.
- Block cache exploits temporal locality: repeated reads hit memory, not disk.

Flash Memory

- Cannot overwrite like RAM/disk sector. Must erase before rewrite.
- Erase is block/granularity larger than page writes.
- Blocks wear out after finite erase cycles.
- FTL maps logical block addresses to physical flash locations.
- Wear leveling spreads erases across device.
- Garbage collection compacts live pages and erases blocks.
- Write amplification: internal writes > user writes.

Flash FTL Details

- Logical page write often goes to fresh physical page; mapping table updated.
- Old physical page becomes invalid/stale, not overwritten in place.
- Garbage collection selects blocks, copies still-live pages elsewhere, erases block.
- Wear leveling may choose less-worn blocks even if not locally optimal for speed.
- FTL hides flash constraints behind disk-like block interface.

Flash Weird Details

- Lecture page header bits: A-W-G can encode allocation/write/garbage state so FTL can recover mapping by scanning headers on startup.
- High utilization makes garbage collection expensive: if erase unit has utilization U, only 1-U becomes free after copying live pages.
- Hot data changes often; cold data rarely changes. Mixing hot+cold is bad because cold live pages get copied repeatedly during GC.
- Sometimes FTL garbage-collects cold units even for little immediate free space to improve wear/locality later.
- TRIM: OS tells SSD logical blocks are no longer useful, reducing needless copying during GC.
- Flash near full often gets slower because GC has less free space and write amplification rises.

Program Memory Layout

- Text/code: instructions, often read-only/shareable.
- Data: initialized globals/statics.
- BSS: zero-initialized globals/statics.
- Heap: dynamic allocation, arbitrary lifetimes.
- Stack: calls, locals, return addresses; per thread.

Linking / Loading

- Compiler -> object files with machine code + symbols + relocation info.
- Linker combines object files, resolves symbols, creates executable.
- Static linking copies library code into executable.
- Dynamic linking uses shared libraries loaded/connected at load/run time.
- Relocation entry: "fix this address later."
- Import table entry: "this external symbol/library function is needed."
- Debug info maps machine code back to source names/lines/types.

Allocator / Heap

- malloc: returns heap block; free: releases it.
- Leak: lost pointer to allocated live block; memory cannot be reused.
- Dangling pointer: pointer to freed/invalid object.
- Double free: freeing same block twice corrupts allocator state.
- Internal fragmentation: waste inside allocated block.
- External fragmentation: free memory split into unusable holes.
- First fit: first adequate hole. Best fit: smallest adequate hole. Coalescing merges adjacent free blocks.

Free Space

- Free list: linked list of free blocks; simple but can have locality/recovery issues.
- Bitmap: one bit per block; easy to find contiguous runs; compact for large disks.
- Allocation policy affects locality and fragmentation.

Virtual Memory

- Each process sees its own virtual address space. MMU translates virtual -> physical and checks permissions.
- Goals: isolation/protection, relocation, sharing, sparse address spaces, demand paging.
 - Virtual address generated by CPU/program. Physical address indexes RAM.
 - Invalid/protected access -> trap/page fault/segfault.

Base and Bound

If $VA < bound$, $PA = base + VA$; else trap.

- Supports relocation/protection for one contiguous segment.
- External fragmentation issue.
- Message-passing style: kernel can translate/copy between process address spaces using each process's base/bound.

Segmentation

- Multiple variable-sized regions, each with base, bound, permissions: code, heap, stack, etc.
- Address = segment number + offset.
 - Check offset against segment bound; add segment base.
 - Allows different permissions/sharing per segment but still external fragmentation.

Paging

Fixed-size virtual pages map to fixed-size physical frames.

$VA = VPN + offset$. Page size = 2^k -> offset is low k bits. $PA = PPN + same\ offset$.

- Page table maps VPN -> PPN + valid/protection/dirty/reference bits.
- No external fragmentation for physical memory; possible internal fragmentation in last page.
- Page table can be huge; use multi-level page tables, TLB.

TLB

- Hardware cache of recent translations.
- TLB hit: fast translation. Miss: consult page table.
- Context switch may require TLB flush or address-space identifiers.

Demand Paging

- Pages loaded into RAM only when used.
- Page fault: page not resident or permission problem. Kernel handles fault.
- On nonresident valid page: choose frame, maybe evict victim, read needed page from disk, update table, restart instruction.
- Dirty victim must be written back before reuse; clean victim can be discarded.

Replacement

policy	idea/trap
Random	simple baseline
FIFO	evict oldest resident; can evict hot pages; Belady anomaly possible
LRU	evict least recently used; good but costly to implement exactly
Clock	approx LRU using reference bit; sweep clears reference bits, evicts unreferenced page

- Local/per-process replacement: victim from faulting process.
- Global replacement: victim from any process; can improve utilization but interference.

Thrashing

System spends most time paging, little time executing. Symptoms: paging disk near 100%, CPU idle/low useful work. Fix by reducing working set, adding memory, or improving paging I/O; scheduling alone usually does not fix.

Address Bit Math

- Page size 2^k -> offset has k bits.
- Virtual address bits = VPN bits + offset bits.
- Physical address bits = PPN bits + offset bits.
- # virtual pages = virtual address space size / page size.
- # physical frames = physical memory size / page size.
- Page table bytes = entries * PTE size, before multi-level savings.

Multi-Level Page Tables

- Split VPN bits into indexes for each level.
- Each level points to next-level page-map page; final level gives PPN/protection.
- Allocate lower-level tables only for used address ranges, saving memory for sparse spaces.
- Entries per page-map page = page size / PTE size.

$VA = [L1_index][L2_index]...[offset]$

TLB Entry Fields

- VPN tag and PPN/frame.
- Valid bit and permission bits.
- Address-space ID/process tag if avoiding flush on context switch.
- May include dirty/reference or cache bits depending design.
- Does not need page offset; offset passes through unchanged.

Page Fault Steps

1. Hardware traps into kernel with faulting VA/reason.
2. OS checks if access is legal in process address space.
3. If illegal: kill/signal process.
4. If legal but nonresident: find free frame or evict victim.
5. If victim dirty, write it to disk/swap.
6. Read needed page into frame; update PTE/TLB; restart instruction.

Worked: Base/Bound Code

```
uintptr_t translate(uintptr_t va) {
    if (va >= bound) trap(); // out of process region
    return base + va;        // physical address
}
```

Message passing still works: sender traps, kernel validates sender pointer/bound, copies bytes into kernel/receiver.

Replacement Simulation

- FIFO: maintain arrival order into memory, not last-use order.
- LRU: update recency on every access, including hits.
- Clock: if ref=1, clear and skip; if ref=0, evict. Hand can move many steps.
- Dirty pages cost write-back; clean pages cheaper to evict.
- MIN/optimal evicts page used farthest in future; only theoretical comparison.

Sharing / Protection

- Two processes can map different VPNs to same PPN for shared memory/code.
- Read-only shared text pages save memory and are safe across processes.
- Copy-on-write lets fork share pages until a write occurs, then copies.
- Permission bits enforce read/write/execute/user access.

Memory Bug Vocabulary

- Buffer overflow: write past allocated object.
- Use-after-free: use pointer after object freed.
- Uninitialized read: value depends on stale memory.
- Stack pointer escape: returning pointer to local stack object.
- Aliasing: two names/pointers refer to same object.

VM / Linking Rapid Fire

- Compile-time: source -> object; link-time: symbols combined/resolved; load-time: executable mapped; run-time: code executes and dynamic linking/page faults may happen.
- Object file can contain unresolved external symbols plus relocation entries.
- Relocation/import tables are metadata for patching or connecting addresses, not ordinary program logic.
- Dynamic linking saves memory/disk when many processes share library text pages.
- Position-independent code helps shared libraries load at flexible addresses.
- Stack grows/shrinks with calls; heap grows/shrinks with allocator; globals live for whole process lifetime.
- Base/bound gives one contiguous protected region; segmentation generalizes to several regions.
- Paging uses fixed sizes, eliminating external fragmentation in physical memory.
- Internal fragmentation still possible inside final page/frame.
- Page fault is not always an error; it can be normal demand paging.
- Protection fault is an illegal access; nonresident valid page is recoverable by loading page.
- Dirty bit affects eviction cost; reference bit helps replacement approximation.
- TLB locality comes from programs repeatedly touching nearby/recent pages.
- Larger pages reduce TLB pressure but may increase internal fragmentation and I/O cost per fault.
- Copy-on-write after fork avoids physically copying all pages until a write.

Translation Worked Mini-Pattern

```
page size = 4096 = 2^12
VA = 0x12345
offset = low 12 bits = 0x345
VPN = VA >> 12 = 0x12
lookup VPN -> PPN
PA = (PPN << 12) | offset
```

Offset never changes during page translation. Only page number changes.

Page Table Design Questions

- Given VA bits and page size, compute VPN bits.
- Given physical memory and page size, compute PPN/frame bits.
- Given PTE size and page size, compute entries per page table page.
- For multi-level design, divide VPN bits into index chunks.
- Include valid/protection/resident bits in PTE; include PPN when resident.

Allocator Free List

- Allocator metadata often stored in block headers/footers.
- Splitting: large free block divided into allocated piece + remaining free block.
- Coalescing: adjacent free blocks merged to reduce external fragmentation.
- Alignment means allocator may round requested sizes upward.

Allocator Weird Details

- **Slab allocator**: allocate large slabs, divide into fixed-size objects, maintain free objects. Fast for common kernel object types; can waste memory if slabs underused.
- **Reference counting**: free when count reaches zero. Fails on cycles; bugs if increments/decrements missed; can still leave dangling pointers.
- **Garbage collection**: runtime finds unreachable objects. Avoids many manual frees but has overhead/pauses and needs language/runtime support.
- **Mark-and-sweep**: mark reachable objects from roots, then sweep unmarked objects into free space.
- Manual memory exam bug distinction: leak = unreachable allocated memory; dangling = pointer to invalid/freed memory.

OS/User Same Address Space

- Some designs map kernel into every process address space for fast syscall/trap transitions.
- User mode cannot access kernel pages because permission bits prohibit it.
- On trap/syscall, CPU switches privilege level and kernel can run using protected mappings.
- Kernel must validate user pointers before copying data to/from user memory.

Device I/O Weird Details

- OS talks to devices through device controllers, not directly to raw hardware mechanics.
- Controllers expose registers: command, status, data/buffer address, block number, etc.
- Programmed I/O: CPU copies data itself; wastes CPU for large transfers.
- DMA: device/controller copies data directly between device and memory; CPU sets up transfer and is interrupted on completion.
- Without interrupts, OS would poll device status; with interrupts, device signals when done.
- Disk read sketch: OS programs controller with block + memory address; controller/DMA transfers; interrupt marks request complete and wakes waiter.

Worked: Address Translation

VA bits 16, page size 1KB, VA 0x2ABC, page table maps VPN 0xA -> PPN 0x35.

```
1KB = 2^10 -> offset 10 bits
offset = 0x2ABC & 0x3FF = 0x2BC
VPN = 0x2ABC >> 10 = 0xA
PA = (0x35 << 10) | 0x2BC
    = 0xD6BC
```

If VPN invalid or permissions fail, trap/page fault instead of producing PA.

Worked: Page Index Extraction

Generic single-level translation for power-of-two page size.

```
int k = log2(page_size);
uint64_t off_mask = (1ULL << k) - 1;
uint64_t offset = va & off_mask;
uint64_t vpn = va >> k;
pte_t pte = page_table[vpn];
if (!pte.valid || !allows(pte.perm, access)) fault();
pa = (pte.ppn << k) | offset;
```

Offset bits are copied unchanged because virtual page and physical frame have same size.

Worked: Multilevel Split

48-bit VA, 4KB pages, 8B PTEs: offset=12 bits, entries/table=4096/8=512=2^9, so 9 bits per level and (48-12)/9=4 levels.

```
offset = va & 0xfff;
L1 = (va >> 36) & 0x1fff;
L2 = (va >> 30) & 0x1fff;
L3 = (va >> 21) & 0x1fff;
L4 = (va >> 12) & 0x1fff;
```

Each index selects an entry in one page-map page; final entry gives the PPN.

Worked: FIFO vs LRU

Frames=3, references: 1 2 3 1 4.

ref	FIFO frames	LRU frames
1	1	1
2	1 2	1 2
3	1 2 3	1 2 3
1	hit; order still 1 oldest	hit; 1 becomes recent
4	evict 1	evict 2

Key difference: LRU updates recency on hits; FIFO does not.

Disk Scheduling		
policy	idea	trap
FIFO	serve in arrival order	bad seek behavior
SSTF	closest request next	may starve far requests
SCAN/elevator	sweep direction, serving requests	edge behavior matters

File API Must-Know		
API	coding meaning	
open(path, O_RDONLY)	get fd for reading.	
open(path, O_WRONLY O_CREAT O_TRUNC, 0644)	create/overwrite output file.	
read(fd, buf, n)	copy bytes from file/pipe/device into memory buffer.	
write(fd, buf, n)	copy bytes from memory buffer to file/pipe/device.	
close(fd)	drop this fd table entry; object persists if other refs.	
unlink(path)	remove directory entry; inode freed when link/open refs gone.	

Assignment 7 Unix V6 APIs		
helper	what it should do	
diskimg_readsector(dfd, s, buf)	read one 512B sector; success == DISKIMG_SECTOR_SIZE.	
inode_iget(fs, inum, &in)	fetch inode; number 1 is inode index 0.	
inode_getsize(&in)	(i_size0<<16) i_size1.	
inode_indexlookup(fs, &in, lbn)	logical file block -> physical sector.	
file_getblock(fs, inum, lbn, buf)	read logical file block; return valid byte count.	
directory_findname(fs, name, dirinum, &de)	scan dir file for name -> dirent.	
pathname_lookup(fs, path)	start at root inode 1; walk components.	

Layering smell: pathname -> directory -> file -> inode -> disk sector.

- Assignment 7 V6 Constants/Structs**
- DISKIMG_SECTOR_SIZE=512, INODE_START_SECTOR=2, ROOT_INUMBER=1, MAX_COMPONENT_LENGTH=14.
 - Disk layout: sector 0 boot, 1 superblock, sector 2.. inode list, then data/indirect/directory sectors.
 - struct inode: i_mode, i_nlink, i_size0/i_size1, i_addr[8].
 - Mode bits: IALLLOC allocated, IFMT type mask, IFDIR directory, ILARG large addressing.
 - struct direntv6: uint16_t d_inumber + char d_name[14]; 14-char name may lack '\0', use strncmp.
 - Derived: 512B sector / 2B pointer = 256 pointers; 512B / 16B dirent = 32 directory entries.

Allocation Strategies		
method	good	bad
contiguous	fast sequential/random, simple	external fragmentation, hard growth
linked	easy growth, no external fragmentation	poor random access, pointer overhead, reliability
FAT	links stored in table; easier traversal/cache	large table, random access still chain-like
indexed/inodes	direct/random access, supports growth	metadata blocks overhead

- Unix V6 / BSD Inodes**
- Inode stores metadata and block pointers. Directory maps name -> inode number.
 - Metadata: type, owner, permissions, size, timestamps, link count, block pointers.
 - Direct pointers: point directly to data blocks.
 - Single-indirect: pointer to block full of data-block pointers.
 - Double-indirect: pointer to block of pointers to indirect blocks.

Block index = file offset / block size. Offset in block = file offset % block size.

For pointer-count problems: pointers per indirect block = block size / pointer size.

- Directories / Links**
- Directory is a file containing mappings from names to inode numbers.
 - Path lookup walks components one directory at a time.
 - Hard link: another directory entry for same inode; link count tracks names.
 - Symbolic link: special file whose contents are another pathname; lookup substitutes target path.
 - Deleting name removes directory entry; inode/data freed when link count zero and no opens.

- Block Cache**
- Cache disk blocks in memory to avoid repeated I/O.
 - Delayed write/write-back improves performance but risks data loss/inconsistency on crash.
 - Synchronous write safer but slower.
 - Dirty block = cache differs from disk and must be written eventually.

- Fragmentation**
- Internal fragmentation: allocated unit larger than data, unused space inside unit.
 - External fragmentation: enough free space exists but split into pieces.
 - BSD FS uses block/fragments ideas to balance large-block throughput and small-file waste.

- BSD Inode Pointer Math**
- Course BSD inode model: 14 block pointers: first 12 direct, 13th single-indirect, 14th double-indirect.
 - If block size B and pointer size P, indirect block holds B/P pointers.
 - Logical blocks 0..11 use direct pointers.
 - Then subtract 12; next B/P logical blocks use single indirect.
 - For double indirect, quotient chooses indirect block, remainder chooses pointer inside it.

- V6 Small/Large Inode Rules**
- If prompt shows i_addr[8] and ILARG, use Assignment 7 V6 rules, not 12-direct BSD rules.
 - Small inode, !(i_mode & ILARG): i_addr[0..7] point directly to data sectors.
 - Large inode: i_addr[0..6] point to singly indirect sectors; i_addr[7] points to doubly indirect sector.
 - Large file first 7*256=1792 logical blocks use single-indirect region.
 - For double: rem=lbn-1792; outer=rem/256; inner=rem%256.

- Path Lookup Example**
- Start at root inode for absolute path or current directory for relative path.
 - Read directory file and find next component name.
 - Directory entry gives inode number.
 - Load that inode; repeat until final component.
 - Open file creates process-local fd pointing to kernel open-file state.
- /a/b/c: lookup a in root, b in a, c in b.

- Open Files and Unlink**
- open returns process-local fd; multiple fds can refer to same inode/open-file state.
 - unlink removes directory entry and decrements link count.
 - If file is still open, data blocks are not necessarily freed until last open reference closes.
 - Filename lives in directory, not usually in inode.

- Filesystem Exam Traps**
- Directory entry maps name to inode; inode does not know all its names.
 - Hard links share inode; symlink is its own inode containing a path.
 - Random access in linked allocation requires chasing pointers.
 - FAT keeps chain pointers in table, so table corruption is serious.
 - Delayed writes improve performance but make crash recovery harder.

Worked: V6 (Large Indirect Block)
inode_iget(fs, inum, &in); size = inode_getsize(&in); first = lbn * DISKIMG_SECTOR_SIZE; if (first >= size) return 0; sector = inode_indexlookup(fs, &in, lbn); diskimg_readsector(fs->dfd, sector, buf); return min(size-first, DISKIMG_SECTOR_SIZE);
Last block may be partial; return valid byte count, not always 512.

Worked: Is Block An Indirect Block?
for each allocated inode; if (!(in.i_mode & ILARG)) continue; for i in 0..6: if (in.i_addr[i] == block) return true; read in.i_addr[7] into ptrs; for each ptr in ptrs: if (ptr == block) return true;

Trap: i_addr[7] itself is the doubly-indirect block; entries inside it name singly indirect blocks.

- Crash-Relevant FS Writes**
- Allocating block changes free map and inode pointer.
 - Creating file changes inode, directory block, free inode map, maybe data block.
 - Appending changes data block, inode size, block pointers, free map.
 - Deleting changes directory entry, link count, free maps, maybe data blocks.
 - Each multi-write operation has intermediate crash states.

Max File Size Formula
For BSD-style 12 direct + 1 indirect + 1 double-indirect: max data blocks = 12 + ptrs + ptrs*ptrs, where ptrs = block_size / pointer_size.
Max bytes = max data blocks * block size. If the problem gives different pointer counts, use the problem's numbers.

- Directory Entry Details**
- Directory contents are ordinary file data interpreted as entries.
 - An entry usually contains name plus inode number.
 - Renaming within same filesystem may only update directory entries, not file data.
 - Moving across filesystems cannot be a pure hard-link rename; data may need copying.
 - Hard link to directory is usually forbidden to prevent cycles confusing traversal/reference counts.

Cache Write Policies		
policy	meaning	risk/cost
write-through	write disk immediately	slow, safer
write-back	mark dirty, write later	fast, crash risk
sync	caller waits for disk	durable but high latency

Worked: Inode Offset
Block size 4KB, pointer 4B, 12 direct. Offset 70KB. ptrs per indirect = 4096/4 = 1024 logical block = 70KB/4KB = 17 17 >= 12, so not direct 17 - 12 = 5 5 < 1024, so single indirect entry 5 byte offset in data block = 70KB % 4KB = 2KB

Answer: use inode's single-indirect pointer, then the 5th pointer inside that indirect block.

Worked: Inode Block Lookup
int block_for(inode *in, int lbn) { int n = BLOCK_SIZE / PTR_SIZE; if (lbn < 12) return in->direct[lbn]; lbn -= 12; if (lbn < n) { block *ind = read_block(in->single); return ind->ptr[lbn]; } lbn -= n; block *dbl = read_block(in->dbl); block *ind = read_block(dbl->ptr[lbn / n]); return ind->ptr[lbn % n]; }
Double-indirect: quotient selects which indirect block; remainder selects entry inside it.

Worked: Path Lookup
Open /usr/bin/ls: root inode -> read root dir, find "usr" usr inode -> read usr dir, find "bin" bin inode -> read bin dir, find "ls" ls inode -> create open-file object process fd table gets new fd -> open-file object
The filename ls is in the bin directory entry, not inside the ls inode.

Worked: Directory Scan
int lookup(inode *dir, char *name) { for each directory entry e in dir's data blocks: if (strcmp(e.name, name) == 0) return e.inumber; return -1; }

Directory file contents map names to inode numbers. Then OS indexes the on-disk inode array/table by that number.

Worked: V6 (inode_indexlookup)
int lookup(fs, inode *in, int b) { uint16_t ptrs[256], ptrs2[256]; if (!(in->i_mode & ILARG)) return in->i_addr[b]; if (b < 7*256) { diskimg_readsector(fs->dfd, in->i_addr[b/256], ptrs); return ptrs[b % 256]; } b -= 7*256; diskimg_readsector(fs->dfd, in->i_addr[7], ptrs); diskimg_readsector(fs->dfd, ptrs[256], ptrs2); return ptrs2[b % 256]; }

Check bounds/zero/error in real code. Trap: i_addr[7] is the doubly-indirect block, not an indirect data-pointer block.

Worked: Scan All V6 Inodes
int n = fs->superblock.s_isize * (DISKIMG_SECTOR_SIZE / sizeof(struct inode)); for (int inum=1; inum<=n; inum++) { struct inode in; if (inode_iget(fs, inum, &in) < 0) return -1; if (!(in.i_mode & IALLLOC)) continue; // test property of allocated inode }

Use this for "find any file/inode/block with property X" final problems.

Worked: Hard Link Skeleton
target = pathname_lookup(fs, oldpath); inode_iget(fs, target, &tin); if (!(tin.i_mode & IFMT)) == IFDIR reject; parent = pathname_lookup(fs, parent_dir(newpath)); find empty dirent slot in parent; slot.d_inumber = target; copy final name into slot.d_name; tin.i_nlink++;

Hard link adds a directory entry to same inode; it does not copy file data.

Worked: Upgrade Small V6 Inode
uint16_t ptrs[256]; for i in 0..7: ptrs[i] = in->i_addr[i]; for i in 8..255: ptrs[i] = 0; clear in->i_addr[0..7]; in->i_addr[0] = new_indirect_sector; in->i_mode = ILARG;
Old data blocks stay put; only the inode's addressing format changes.

Crash Recovery Problem

Filesystem updates often require multiple disk writes. Crash between writes can leave metadata inconsistent.

- **Data loss:** recent user data missing but filesystem structure ok.
- **Inconsistency:** metadata contradicts itself; worse.
- Atomic operation: appears all-or-nothing.
- Durability: once reported done, survives crash.
- Consistency: filesystem invariants hold.

Common Inconsistencies

- Inode points to block marked free.
- Block allocated to two files.
- Directory entry points to missing/free inode.
- Link count does not match directory references.
- Free bitmap/list disagrees with inodes.
- Cached dirty metadata lost while data written, or vice versa.

Goals

- Post-crash checker scans filesystem and repairs inconsistencies.
- Can be slow; needs whole-structure scan.
- May preserve consistency but not necessarily recover newest intended data.
- Repairs should be idempotent: safe to repeat after another crash.

Ordered Writes

Write blocks in a safe order so every crash point leaves recoverable/acceptable state.

- Example allocation: initialize data/inode before making directory entry visible.
- Can trade data loss for metadata consistency.
- Hard to get right for complex multi-block updates.

Write-Ahead Logging

1. Write intended updates to log.
2. Commit log transaction.
3. Apply updates to real filesystem locations.
4. Checkpoint/truncate log when safe.

- Recovery: committed transaction replayed; uncommitted ignored.
- Write-ahead rule: log reaches disk before home-location updates.
- Transaction groups updates into all-or-nothing unit.

Virtual Machines

- VM virtualizes whole hardware-like machine, not just process address space.
- Guest OS runs inside VM; hypervisor/VMM manages real hardware.
- Benefits: isolation, consolidation, portability, snapshots.
- Challenges: privileged instructions, device virtualization, memory virtualization, performance.

Trust / Ethics

- Trust = accepting vulnerability to another actor/system because you expect acceptable behavior.
- Agency: who can act/decide? Who is affected?
- Trust by assumption: assume system/person is trustworthy.
- Trust by inference: infer from evidence/history/signals.
- Trust by substitution: trust intermediary/certifier/tool instead.
- Over-trust: relying beyond warranted evidence/capability.
- Discuss stakeholders, pervasiveness, time/long-term effects, incentives, accountability.

Final-Code Traps

- After `fork`, both parent and child continue from next instruction.
- `execvp` success means no old-code lines after it run.
- FD redirection before `exec` survives into new program.
- Pipe EOF requires all write ends closed.
- `waitpid` waits for process, not thread.
- Threads share globals/heap/open files; processes after fork do not share ordinary memory.
- User-level thread metadata lives in process memory, so fork copies it as data.
- Condition variable waits must be in `while`.
- Interrupt disabling is internal/brief; user mutex does not disable interrupts for whole critical section.

Address Translation Checklist

1. Identify page/segment size.
2. Split virtual address into VPN/offset or segment/offset.
3. Check valid/protection/bound.
4. Map VPN->PPN or add base.
5. Keep same offset.
6. If invalid/not resident: page fault/trap.

Inode Offset Checklist

1. `logical_block = offset / block_size`.
2. `inside = offset % block_size`.
3. If logical block in direct range, use direct pointer.
4. Else subtract direct count; check single indirect capacity.
5. Else subtract single capacity; compute double-indirect indexes.
6. Pointers per indirect block = block size / pointer size.

One-Line Exam Answers

- Race? Shared mutable state accessed without sufficient synchronization.
- Deadlock? Cycle of waiting with no preemption/order violation.
- Thrashing? Working sets exceed memory, paging dominates.
- BSD priority? Recent CPU use lowers priority; waiting/blocking improves it.
- Dynamic linking? Shared library symbols resolved/loaded at load/run time.
- Crash consistency? Need ordered writes/logging/fsck to preserve metadata invariants.

Create File Crash Points

Typical writes: allocate inode, initialize inode, allocate data block, write data, update inode pointers/size, add directory entry, update free map/link count.

- Dir entry before initialized inode -> name points to garbage.
- Inode pointer before data initialized -> file exposes old garbage block contents.
- Free map says block free while inode points to it -> future double allocation.
- Allocated but unreachable block -> leak; less dangerous than double allocation.

Logging Recovery Logic

- No commit record: ignore transaction; it was not promised durable.
- Commit present: redo/replay updates until home locations reflect transaction.
- Redo must be idempotent because crash can happen during recovery.
- Checkpoint means logged updates have reached home locations, so log can be truncated/reused.

Ordered Write Rule Examples

- Write new data block before inode points to it.
- Write inode before directory entry makes name reachable.
- For deletion, remove directory entry before freeing inode/blocks.
- Goal is usually metadata consistency first, even if newest user data can be lost.

VM Implementation Terms

- Trap-and-emulate: guest privileged operation traps to hypervisor; hypervisor emulates effect.
- Direct execution: safe ordinary instructions run directly on CPU for speed.
- Shadow page tables: hypervisor-maintained mappings from guest virtual to machine physical when hardware support limited.
- Nested paging/EPT: hardware helps translate guest virtual -> guest physical -> machine physical.

VM Weird Details

- **Hosted VMM:** runs on top of a conventional host OS; VMM is an app/service, host OS controls real hardware.
- **Full simulation:** simulate CPU/memory/devices/traps; very flexible but slow.
- **Direct execution:** ordinary guest instructions run on real CPU; privileged/sensitive operations trap.
- Guest CLI: should disable virtual interrupts for that virtual CPU, not real machine interrupts for everyone.
- Virtual I/O devices: guest reads/writes virtual device registers; hypervisor simulates device and injects virtual interrupt on completion.
- Three address spaces idea: guest virtual, guest physical, machine physical.

Trust Answer Skeleton

1. State the trust relationship: who trusts whom/what, for what action.
2. Identify stakeholders, including indirect affected parties.
3. Separate capability, intent, evidence, incentives, accountability.
4. Name trust type: assumption, inference, substitution, or over-trust.
5. Discuss pervasiveness/time: scale, lock-in, future effects, reversibility.

High-Yield Definitions

- Idempotent: doing it multiple times has same effect as once.
- Working set: pages actively needed by process in recent window.
- Root of trust: component assumed trustworthy because other trust depends on it.
- Reference bit: hardware/software bit indicating page was accessed.
- Dirty bit: page/block modified since loaded; must be written back before eviction.

When Asked "Why?"

- Why locks? Mutual exclusion over shared mutable state.
- Why CVs? Sleep without busy-waiting until predicate may change.
- Why paging? Fixed-size allocation, protection, sparse spaces, demand loading.
- Why TLB? Page-table walks are expensive; cache translations.
- Why logging? Multi-write updates need crash-atomic appearance.
- Why FTL? Flash erase/write/wear constraints differ from disk interface.

Crash / Trust Rapid Fire

- Crash can happen after any disk write unless problem states atomic group.
- Disk write completion order may differ from issue order unless forced/ordered by system.
- Journaling/logging converts many home-location writes into a recoverable transaction protocol.
- Redo log records after-images; undo log records old values; course mostly needs high-level WAL idea.
- Commit record is the boundary between "must replay" and "ignore".
- Inconsistent free map can allocate same block twice or leak a block.
- Idempotent recovery matters because crash can occur during recovery itself.
- VM isolation depends on hypervisor correctness; guest OS is not fully trusted by host.
- Virtualization extends the OS idea: provide an abstraction that looks like hardware.
- Trust by substitution: trusting compiler/package manager/certificate authority instead of personally verifying all code.
- Over-trust example: assuming software is safe because it is popular or polished.
- Ethics answers should name concrete stakeholders and harms, not just "users".
- Agency question: who can consent, override, inspect, or contest the system's actions?

Past-Final Short-Answer Traps

- **stack_switch**: switching T1 -> T2 returns using T2's saved return address; T1's return address is used when switching back.
- **close-on-exec bit**: belongs in descriptor-table entry, not open-file table entry, because two fds to same file may differ.
- **Privileged?** syscall trap: unprivileged: return-from-kernel, set page-map base, disable interrupts: privileged; TLB invalidate/jump-table fill can be unprivileged.
- **Linker?** combines sections, creates symbol table, modifies code/data relocation addresses, chooses segment addresses; dynamic loader fills lazy import/jump table.
- **Condition variables**: copying is invalid because wait queues would be duplicated/split nonsensically.
- **MRU vs LRU**: MRU can beat LRU on looping pattern like ABCDABCD with 3 frames.
- **Agential gullibility**: agent/company trusts another agent/system beyond justified evidence, e.g. removing safeguards and skipping QA because prior project seemed reliable.
- **FAT math**: 16-bit FAT -> at most 2^{16} blocks; 512B blocks -> 2^{25} B disk; FAT memory $2^{16} * 2 = 2^{17}$ B.
- **PML entry address**: physical frame from previous-level entry; offset = relevant 9-bit index shifted left 3 because each entry is 8B.
- **V6 erase front**: erase exactly one 512B block -> shift pointers/modify indirect blocks; erase less than block -> rewrite data blocks.

Recovery Method Comparison

method	strength	cost/trap
fsck	repairs after crash	slow, may lose recent data
ordered writes	simple consistency discipline	hard for complex updates
logging	transactional recovery	extra writes/log management

Worked: Crash-Safe Update Skeleton

```
// redo logging shape
write_log("begin T");
write_log("new bitmap block");
write_log("new inode block");
write_log("commit T"); flush_log();
install_bitmap_block to home_location;
install_inode_block to home_location;
clear/checkpoint_log;
```

- Crash before commit: ignore log; old filesystem should remain valid.
- Crash after commit: replay all logged updates, even if some home writes happened.
- Replay must be idempotent: same committed update twice gives same final state.

Final Strategy

- For code traces: write process/thread identities and shared vs copied state first.
- For monitor design: state variables + wait predicates before code.
- For address translation: compute offset bits first.
- For filesystem math: convert byte offset to logical block number first.
- For crash recovery: list writes, then examine crash after each write.
- For ethics: identify trust relationship and stakeholders explicitly.

Coding Section Coverage Checklist

- Trace `fork` values/process count/output order; remember copied memory.
- Write shell code: `fork`, `execvp`, `waitpid`, redirection, pipe, close unused FDs.
- Use `argv/char *args[]` with final NULL; distinguish string `"\0"`.
- Trace fd table/open-file sharing, `dup2` direction, pipe EOF behavior.
- Fix shared-state race with mutex/RAII; explain why `x++` is not atomic.
- Write monitor code: state fields, lock, `while` predicate, wait, update, notify.
- Handle selected waiters/grouping with queues/records, not just counters.
- Spot deadlock/starvation and add lock ordering/fairness/generation state.
- Compute base/bound, page offset/VPN/PPN, multilevel indexes, replacement traces.
- Use Assignment 7 V6 APIs for inode scans, dir scans, path lookup, hard links, indirect blocks.
- List crash points and decide leak vs dangling pointer vs inconsistent metadata.

Worked: Crash Ordering

Append one new block to file. Needed writes: data block D, inode pointer/size I, free bitmap B.

bad order	crash consequence
I before D	file may point at uninitialized/old garbage data
I before B	inode uses block that free map may later allocate elsewhere
B before I	allocated block may be leaked if crash before inode points to it

Often acceptable order prefers leak over inconsistency: write D, mark/prepare allocation, then make inode reference visible, with recovery/logging to clean up.

Worked: Log Recovery

```
log: [update A] [update B] [COMMIT]
```

Commit present -> replay A and B to home locations, even if crash happened halfway through applying. No commit -> ignore. Replay must be idempotent.

```
log: [update A] [update B]    // no commit
```

Ignore because transaction was not committed/durable.

Worked: Trust Answer

Prompt: "Should users trust an auto-update system?"

1. Trust relation: user trusts vendor/updater to install correct code.
2. Evidence: signatures, vendor history, reproducible builds, audits.
3. Substitution: user trusts signing keys/package infrastructure instead of inspecting code.
4. Risks: compromised key, malicious update, dependency attack, forced changes.
5. Stakeholders: users, admins, affected third parties, vendor, attackers.

Trust Weird Details

- **xz/ssh attack**: supply-chain trust failure; trusted maintainer/project path was used to introduce malicious behavior.
- **Confirmation bias**: people trust evidence more when it matches expectations; platforms can profit from it.
- **False trust in numbers**: quantitative output can look objective even when measurement/model is flawed.
- **Generative AI**: easy to over-trust because fluent output sounds authoritative; verify against primary logs/tests/sources.
- **Synthetic media**: photos/video/audio used to imply authenticity because fabrication was hard; deepfakes weaken that inference.
- Linux trust examples: users trust Linux through reputation/distribution/community; developers trust via review, maintainers, tests, signed releases, social/process evidence.